

scalability-results

Matchmaking scalability test

Machine: Apple M1 Max, 10 cores, 32 GB RAM, Node v26.4.0, socket.io 4.8.3, Redis 8.6.3 (dev container `codeclash-redis-dev`).

What was tested

The matchmaking step of the match flow: a player connects over Socket.IO, emits `join_match_queue`, and waits for `users_matched`. The backend's real `joinMatchQueue` handler, real `notifyMatchFound` and real `MatchConfirmationService` run on a real `socket.io` server, and the players are real `socket.io-client` connections. The only stub is `user_repo.getUserData`, so no Postgres is needed. Each run starts one process holding both the server and all the clients.

Two variants:

- **Stub matchmaking (`--mode=stub`)**: an in-memory matcher that pairs players in join order. This isolates the socket layer and is the same method as the 13 Sept proof of concept, which measured 200 pairs with p50 162 ms and p95 262 ms.
- **Real matchmaking on Redis (`--mode=redis`)**: the real `MatchmakingService` and `MatchmakingCache` against the dev Redis, isolated in **database 15**. The app uses database 0. The script empties database 15 before and after each run.

Every player joins with elo 1400 in `math` mode, so any two players are a valid match. `--stagger` sets the gap between successive joins: 0 means everyone joins at once, 1 ms is about 1,000 joins/s, and 5 ms is about 200 joins/s. A player counts as **stranded** if no `users_matched` arrives within the timeout. Each configuration was run **3 times**.

How to run it

1. Save the script below as `backend/src/scalability/matchmaking-load.ts`. It imports `socket.io-client`, which resolves from the repo-root `node_modules`, and `ioredis`, which the backend already depends on.
2. For `--mode=redis`, the dev Redis must be up (`docker ps` should show `codeclash-redis-dev` on port 6380). Stub mode needs nothing running.
3. Run from `backend/` :

```
cd backend
export TS_NODE_TRANSPILE_ONLY=1

# socket layer only
npx ts-node -r tsconfig-paths/register src/scalability/matchmaking-load.ts
```

```
--mode=stub --players=200
```

```
# real matchmaking on Redis db 15, everyone joins at once / ~1000 joins/s  
/ ~200 joins/s
```

```
npx ts-node -r tsconfig-paths/register src/scalability/matchmaking-load.ts
```

```
--mode=redis --players=200 --stagger=0 --timeout=3000
```

```
npx ts-node -r tsconfig-paths/register src/scalability/matchmaking-load.ts
```

```
--mode=redis --players=200 --stagger=1 --timeout=3000
```

```
npx ts-node -r tsconfig-paths/register src/scalability/matchmaking-load.ts
```

```
--mode=redis --players=200 --stagger=5 --timeout=3000
```

Each run prints one JSON line. The fields are:

Field	Meaning
connect_all_ms	time for all clients to finish the WebSocket handshake
queue_wait_ms	from the first join to the last player either being matched or timing out
matched / stranded	players that did / did not receive <code>users_matched</code> before the timeout
groups	distinct <code>group_id</code> s created (correct value = <code>players / 2</code>)
players_in_multiple_groups	players placed in more than one match (should be 0)
duplicate_events	players who received <code>users_matched</code> more than once
left_in_redis_queue	entries still in the Redis <code>match</code> queue at the end
p50/p95/p99/max_ms	time from a player's join to their first <code>users_matched</code>
rss_mb	memory of the one process holding server and clients

The sweep used here:

```
for n in 20 100 200; do for r in 1 2 3; do npx ts-node -r tsconfig-  
paths/register src/scalability/matchmaking-load.ts --mode=stub --  
players=$n --timeout=20000; done; done  
for st in 0 1 5; do for n in 20 100 200; do for r in 1 2 3; do npx ts-node  
-r tsconfig-paths/register src/scalability/matchmaking-load.ts --  
mode=redis --players=$n --stagger=$st --timeout=3000; done; done; done
```

Results

1. Socket layer (stub matchmaking): scales cleanly

Medians of 3 runs:

Players	Matched	Stranded	Connect all (ms)	p50 (ms)	p95 (ms)	Max (ms)	RSS (MB)
20	60/60	0	34.8	13.8	14	14	189
100	300/300	0	71.8	18.1	18.8	18.9	200
200	600/600	0	113.7	24.7	26.2	26.4	208

Every player was matched in every run, with no duplicates and no server errors. Latency grows gently with load, and p95 stays within a few ms of p50, so the event loop isn't queueing. The socket layer is not the bottleneck at this scale.

2. Real matchmaking on Redis: correctness breaks under concurrency

Totals across 3 runs for each row:

Join rate	Players	Matched	Stranded	Double-matched players	Groups (expected)	p50 (ms)	p95 (ms)
all at once	20	0/60	60	0	0/0/0 (10)	—	—
all at once	100	0/300	300	0	0/0/0 (50)	—	—
all at once	200	0/600	600	0	0/0/0 (100)	—	—
~1000/s (1 ms)	20	57/60	3	9	16/16/16 (10)	15	23.9
~1000/s (1 ms)	100	298/300	2	23	60/61/58 (50)	2.2	20.2
~1000/s (1 ms)	200	599/600	1	58	114/117/116 (100)	2.2	14.7
~200/s (5 ms)	20	60/60	0	0	10/10/10 (10)	8.6	19.4
~200/s (5 ms)	100	298/300	2	2	50/50/50 (50)	7.4	13.3
~200/s (5 ms)	200	598/600	2	2	100/100/100 (100)	7.8	10.6

No run raised a server error, so every failure is silent.

Findings

1. **Players who join at the same moment are never matched.** At 0 ms stagger, 0 of the players were matched at every size, and they all stayed in the Redis queue.
`MatchmakingService.matchmaking`
(`backend/src/application/usecases/services/matchmaking.service.ts:26`) reads the queue, then decides, then writes, all as separate Redis calls. Each concurrent joiner reads an empty queue and adds themselves. Nothing re-runs matchmaking for players already waiting: there's no timer in `server.ts`, and `match_attempt` only changes on the next join or decline. So those players wait forever unless someone else joins later.
2. **At high join rates, one player can be matched into two games.** At 1 ms stagger, 3–21 players per run were placed in two groups, and the group count was above `players/2` (for example 114–117 groups for 200 players, against an expected 100). Two concurrent joiners can both choose the same waiting candidate before either one removes it with `deleteUser`. That player then receives `users_matched` twice, and two different opponents are paired with the same person.
3. **At about 200 joins/s it is mostly correct, but not fully.** At 5 ms stagger, 20 players were always clean. At 100–200 players, 2 of the 3 runs still stranded one player and double-matched one.
4. **Speed is not the problem.** When matching succeeds, p95 is about 10–25 ms. The failure is correctness under concurrent joins, not throughput.

Likely direction for a fix, not implemented here: make the search-and-remove step atomic, for example a Lua script or `WATCH / MULTI` around the queue read, `ZREM` and `HDEL`. A lighter option is to serialise `matchmaking()` per queue in-process, which only works while there's a single backend instance. Separately, a periodic re-scan of waiting players would rescue the stranded ones.

Limits of this test

- Server and clients run in one Node process on one machine, so client CPU competes with the server's. The real backend runs in Docker with its own event loop.
- Only the matchmaking step is covered. Accepting a match, starting the game, submitting answers and marking (Judge0) are not load-tested.
- Socket auth is bypassed: the `user_id` comes from the handshake, not a Cognito JWT, and `user_repo` is stubbed.
- Every player has the same elo, which is the worst case for contention because everyone competes for the same candidates.

Script: `backend/src/scalability/matchmaking-load.ts`

This is the exact file that produced the results. `npm run tsc --noEmit` in `backend/` is clean with it in place.

```
import { createServer } from 'node:http';
import type { AddressInfo } from 'node:net';
```

```

import { performance } from 'node:perf_hooks';
import Redis from 'ioredis';
import { Server as IOServer } from 'socket.io';
import { io as ioClient, type Socket as ClientSocket } from 'socket.io-client';

import { joinMatchQueue } from 'src/interface-adapters/socket-handlers/matchmaking-handlers';
import { MatchConfirmationService } from 'src/application/usecases/services/match/match-confirmation.service';
import { MatchmakingService } from 'src/application/usecases/services/matchmaking.service';
import { MatchmakingCache } from 'src/interface-adapters/cache/matchmaking-cache';

// --mode=stub : in-memory matcher (socket layer only, same method as the
// Sept 13 proof of concept)
// --mode=redis : real MatchmakingService + MatchmakingCache on Redis db
// 15 (isolated from app data in db 0)
const arg = (k: string, d: string) => process.argv.find((a) =>
a.startsWith(`--${k}=`)).split('=')[1] ?? d;
const MODE = arg('mode', 'stub');
const PLAYERS = Number(arg('players', '100'));
const TIMEOUT_MS = Number(arg('timeout', '5000'));
const STAGGER_MS = Number(arg('stagger', '0')); // gap between successive
joins, 0 = everyone at once

function stubMatchmaking() {
  const waiting: { id: string; elo: number }[] = [];
  return {
    matchmaking: async (user: { id: string; elo: number }) => {
      const other = waiting.shift();
      if (other) return [{ id: user.id, elo: user.elo }, other];
      waiting.push({ id: user.id, elo: user.elo });
      return null;
    },
  } as any;
}

async function main() {
  const redis = MODE === 'redis' ? new Redis({ host: 'localhost', port:
6380, db: 15 }) : null;
  if (redis) await redis.flushdb(); // db 15 is scratch space for this
script only
  const matchmaking_service = redis ? new MatchmakingService(new
MatchmakingCache(redis)) : stubMatchmaking();
  const confirmation = new MatchConfirmationService();
  const user_repo = { getUserData: async (id: string) => ({ username: id
}) } as any;

```

```

const httpServer = createServer();
const io = new IO Server(httpServer, { transports: ['websocket'] });
io.on('connection', (socket) => {
  socket.data = { user_id: socket.handshake.auth.user_id };
  socket.on('join_match_queue', (data) =>
    joinMatchQueue(io, socket, data, matchmaking_service, confirmation,
user_repo).catch((e: Error) => server_errors.push(e.message)),
  );
});
await new Promise<void>((r) => httpServer.listen(0, r));
const port = (httpServer.address() as AddressInfo).port;
const server_errors: string[] = [];

const connect = (id: string) => new Promise<ClientSocket>((resolve,
reject) => {
  const s = ioClient(`http://localhost:${port}`, { auth: { user_id: id
}, transports: ['websocket'], reconnection: false });
  s.once('connect', () => resolve(s));
  s.once('connect_error', reject);
});

const connectStart = performance.now();
const sockets = await Promise.all(Array.from({ length: PLAYERS }, (_, i)
=> connect(`load-${i}`)));
const connectMs = performance.now() - connectStart;

const received = new Map<string, number>();
const groups = new Map<string, Set<string>>();
const times: number[] = [];

const queueStart = performance.now();
const outcomes = await Promise.all(sockets.map((s, i) => new
Promise<boolean>((resolve) => {
  const id = `load-${i}`;
  const start = performance.now() + i * STAGGER_MS;
  const timer = setTimeout(() => resolve(false), TIMEOUT_MS + i *
STAGGER_MS);
  s.on('users_matched', (res: { group_id: string; players: { id: string
}[] }) => {
    received.set(id, (received.get(id) ?? 0) + 1);
    const g = groups.get(res.group_id) ?? new Set();
    res.players.forEach((p) => g.add(p.id));
    groups.set(res.group_id, g);
    if (received.get(id) === 1) {
      times.push(performance.now() - start);
      clearTimeout(timer);
      resolve(true);
    }
  });
  setTimeout(() => s.emit('join_match_queue', { elo: 1400, match_mode:

```

```

'math' })), i * STAGGER_MS);
    }));
    const queueMs = performance.now() - queueStart;
    await new Promise((r) => setTimeout(r, 300)); // catch late duplicate
events

    const inGroups = new Map<string, number>();
    for (const g of groups.values()) for (const id of g) inGroups.set(id,
(inGroups.get(id) ?? 0) + 1);

    const sorted = [...times].sort((a, b) => a - b);
    const pct = (p: number) => sorted.length ?
Number(sorted[Math.min(sorted.length - 1, Math.floor((p / 100) *
sorted.length))]!.toFixed(1)) : null;
    const leftInQueue = redis ? await redis.zcard('math') : null;

    console.log(JSON.stringify({
      mode: MODE, players: PLAYERS, stagger_ms: STAGGER_MS,
      connect_all_ms: Number(connectMs.toFixed(1)),
      queue_wall_ms: Number(queueMs.toFixed(1)),
      matched: outcomes.filter(Boolean).length,
      stranded: outcomes.filter((o) => !o).length,
      groups: groups.size,
      players_in_multiple_groups: [...inGroups.values()].filter((n) => n >
1).length,
      duplicate_events: [...received.values()].filter((n) => n > 1).length,
      left_in_redis_queue: leftInQueue,
      server_errors: server_errors.length,
      p50_ms: pct(50), p95_ms: pct(95), p99_ms: pct(99), max_ms:
sorted.length ? Number(sorted[sorted.length - 1]!.toFixed(1)) : null,
      rss_mb: Math.round(process.memoryUsage().rss / 1048576),
    }));

    sockets.forEach((s) => s.disconnect());
    io.close();
    if (redis) { await redis.flushdb(); redis.disconnect(); }
    process.exit(0);
  }

  main();

```

Raw output (evidence)

Every run from the sweep above, one JSON line each, exactly as printed.

Stub mode

```

{"mode":"stub","players":20,"stagger_ms":0,"connect_all_ms":34.8,"queue_wa
ll_ms":14.1,"matched":20,"stranded":0,"groups":10,"players_in_multiple_gro

```

```

ups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors":0,"
p50_ms":13.8,"p95_ms":14,"p99_ms":14,"max_ms":14,"rss_mb":189}
{"mode":"stub","players":20,"stagger_ms":0,"connect_all_ms":32.6,"queue_wa
ll_ms":14.1,"matched":20,"stranded":0,"groups":10,"players_in_multiple_gro
ups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors":0,"
p50_ms":13.8,"p95_ms":14,"p99_ms":14,"max_ms":14,"rss_mb":189}
{"mode":"stub","players":20,"stagger_ms":0,"connect_all_ms":34.8,"queue_wa
ll_ms":14.3,"matched":20,"stranded":0,"groups":10,"players_in_multiple_gro
ups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors":0,"
p50_ms":13.9,"p95_ms":14.1,"p99_ms":14.1,"max_ms":14.1,"rss_mb":189}
{"mode":"stub","players":100,"stagger_ms":0,"connect_all_ms":71.8,"queue_w
all_ms":20.5,"matched":100,"stranded":0,"groups":50,"players_in_multiple_g
roups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors":0
,"p50_ms":19.3,"p95_ms":20.1,"p99_ms":20.1,"max_ms":20.1,"rss_mb":200}
{"mode":"stub","players":100,"stagger_ms":0,"connect_all_ms":71.4,"queue_w
all_ms":19.2,"matched":100,"stranded":0,"groups":50,"players_in_multiple_g
roups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors":0
,"p50_ms":18.1,"p95_ms":18.8,"p99_ms":18.9,"max_ms":18.9,"rss_mb":201}
{"mode":"stub","players":100,"stagger_ms":0,"connect_all_ms":72,"queue_wal
l_ms":18.6,"matched":100,"stranded":0,"groups":50,"players_in_multiple_gro
ups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors":0,"
p50_ms":17.5,"p95_ms":18.3,"p99_ms":18.3,"max_ms":18.3,"rss_mb":199}
{"mode":"stub","players":200,"stagger_ms":0,"connect_all_ms":113.7,"queue_
wall_ms":26.5,"matched":200,"stranded":0,"groups":100,"players_in_multiple
_groups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors"
:0,"p50_ms":24.4,"p95_ms":26,"p99_ms":26.1,"max_ms":26.1,"rss_mb":208}
{"mode":"stub","players":200,"stagger_ms":0,"connect_all_ms":138,"queue_wa
ll_ms":27,"matched":200,"stranded":0,"groups":100,"players_in_multiple_gro
ups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors":0,"
p50_ms":24.9,"p95_ms":26.5,"p99_ms":26.6,"max_ms":26.6,"rss_mb":208}
{"mode":"stub","players":200,"stagger_ms":0,"connect_all_ms":111.5,"queue_
wall_ms":26.7,"matched":200,"stranded":0,"groups":100,"players_in_multiple
_groups":0,"duplicate_events":0,"left_in_redis_queue":null,"server_errors"
:0,"p50_ms":24.7,"p95_ms":26.2,"p99_ms":26.3,"max_ms":26.4,"rss_mb":208}

```

Redis mode

```

{"mode":"redis","players":20,"stagger_ms":0,"connect_all_ms":25.9,"queue_w
all_ms":3004,"matched":0,"stranded":20,"groups":0,"players_in_multiple_gro
ups":0,"duplicate_events":0,"left_in_redis_queue":20,"server_errors":0,"p5
0_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":189}
{"mode":"redis","players":20,"stagger_ms":0,"connect_all_ms":25.5,"queue_w
all_ms":3002.1,"matched":0,"stranded":20,"groups":0,"players_in_multiple_g
roups":0,"duplicate_events":0,"left_in_redis_queue":20,"server_errors":0,"
p50_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":189}
{"mode":"redis","players":20,"stagger_ms":0,"connect_all_ms":26.4,"queue_w
all_ms":3001.8,"matched":0,"stranded":20,"groups":0,"players_in_multiple_g
roups":0,"duplicate_events":0,"left_in_redis_queue":20,"server_errors":0,"
p50_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":189}

```

```
{"mode":"redis","players":100,"stagger_ms":0,"connect_all_ms":62,"queue_wa  
ll_ms":3001.2,"matched":0,"stranded":100,"groups":0,"players_in_multiple_g  
roups":0,"duplicate_events":0,"left_in_redis_queue":100,"server_errors":0,  
"p50_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":197}  
{"mode":"redis","players":100,"stagger_ms":0,"connect_all_ms":60.8,"queue_  
wall_ms":3002.1,"matched":0,"stranded":100,"groups":0,"players_in_multiple_  
groups":0,"duplicate_events":0,"left_in_redis_queue":100,"server_errors":  
0,"p50_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":198}  
{"mode":"redis","players":100,"stagger_ms":0,"connect_all_ms":62.9,"queue_  
wall_ms":3001.2,"matched":0,"stranded":100,"groups":0,"players_in_multiple_  
groups":0,"duplicate_events":0,"left_in_redis_queue":100,"server_errors":  
0,"p50_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":199}  
{"mode":"redis","players":200,"stagger_ms":0,"connect_all_ms":109,"queue_w  
all_ms":3001.1,"matched":0,"stranded":200,"groups":0,"players_in_multiple_  
groups":0,"duplicate_events":0,"left_in_redis_queue":200,"server_errors":0  
,"p50_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":205}  
{"mode":"redis","players":200,"stagger_ms":0,"connect_all_ms":103.4,"queue_  
wall_ms":3002.5,"matched":0,"stranded":200,"groups":0,"players_in_multipl  
e_groups":0,"duplicate_events":0,"left_in_redis_queue":200,"server_errors"  
:0,"p50_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":205}  
{"mode":"redis","players":200,"stagger_ms":0,"connect_all_ms":140.8,"queue_  
wall_ms":3002.9,"matched":0,"stranded":200,"groups":0,"players_in_multipl  
e_groups":0,"duplicate_events":0,"left_in_redis_queue":200,"server_errors"  
:0,"p50_ms":null,"p95_ms":null,"p99_ms":null,"max_ms":null,"rss_mb":205}  
{"mode":"redis","players":20,"stagger_ms":1,"connect_all_ms":25.1,"queue_w  
all_ms":3004,"matched":19,"stranded":1,"groups":16,"players_in_multiple_gr  
oups":3,"duplicate_events":3,"left_in_redis_queue":1,"server_errors":0,"p5  
0_ms":14.9,"p95_ms":23.9,"p99_ms":23.9,"max_ms":23.9,"rss_mb":192}  
{"mode":"redis","players":20,"stagger_ms":1,"connect_all_ms":24.8,"queue_w  
all_ms":3005.8,"matched":19,"stranded":1,"groups":16,"players_in_multiple_  
groups":3,"duplicate_events":3,"left_in_redis_queue":1,"server_errors":0,"  
p50_ms":15,"p95_ms":23.3,"p99_ms":23.3,"max_ms":23.3,"rss_mb":189}  
{"mode":"redis","players":20,"stagger_ms":1,"connect_all_ms":24.9,"queue_w  
all_ms":3004.5,"matched":19,"stranded":1,"groups":16,"players_in_multiple_  
groups":3,"duplicate_events":3,"left_in_redis_queue":1,"server_errors":0,"  
p50_ms":15.5,"p95_ms":24.8,"p99_ms":24.8,"max_ms":24.8,"rss_mb":193}  
{"mode":"redis","players":100,"stagger_ms":1,"connect_all_ms":60.8,"queue_  
wall_ms":100.6,"matched":100,"stranded":0,"groups":60,"players_in_multiple_  
groups":8,"duplicate_events":8,"left_in_redis_queue":0,"server_errors":0,  
"p50_ms":2.5,"p95_ms":16.5,"p99_ms":24,"max_ms":24,"rss_mb":201}  
{"mode":"redis","players":100,"stagger_ms":1,"connect_all_ms":60.8,"queue_  
wall_ms":3099.2,"matched":99,"stranded":1,"groups":61,"players_in_multiple_  
groups":9,"duplicate_events":9,"left_in_redis_queue":1,"server_errors":0,  
"p50_ms":2,"p95_ms":25.1,"p99_ms":28,"max_ms":28,"rss_mb":201}  
{"mode":"redis","players":100,"stagger_ms":1,"connect_all_ms":61.5,"queue_  
wall_ms":3100.4,"matched":99,"stranded":1,"groups":58,"players_in_multiple_  
groups":6,"duplicate_events":6,"left_in_redis_queue":1,"server_errors":0,  
"p50_ms":2.2,"p95_ms":20.2,"p99_ms":23.2,"max_ms":23.2,"rss_mb":202}  
{"mode":"redis","players":200,"stagger_ms":1,"connect_all_ms":141,"queue_w  
all_ms":201.1,"matched":200,"stranded":0,"groups":114,"players_in_multiple
```

```
_groups":16,"duplicate_events":16,"left_in_redis_queue":0,"server_errors":0,"p50_ms":1.8,"p95_ms":14.6,"p99_ms":23.2,"max_ms":23.3,"rss_mb":210}
{"mode":"redis","players":200,"stagger_ms":1,"connect_all_ms":111.8,"queue_wall_ms":201.7,"matched":200,"stranded":0,"groups":117,"players_in_multiple_groups":21,"duplicate_events":21,"left_in_redis_queue":0,"server_errors":0,"p50_ms":2.4,"p95_ms":16.1,"p99_ms":25.5,"max_ms":25.5,"rss_mb":208}
{"mode":"redis","players":200,"stagger_ms":1,"connect_all_ms":106.8,"queue_wall_ms":3199.6,"matched":199,"stranded":1,"groups":116,"players_in_multiple_groups":21,"duplicate_events":21,"left_in_redis_queue":1,"server_errors":0,"p50_ms":2.2,"p95_ms":14.7,"p99_ms":24.3,"max_ms":24.4,"rss_mb":212}
{"mode":"redis","players":20,"stagger_ms":5,"connect_all_ms":25.6,"queue_wall_ms":97.7,"matched":20,"stranded":0,"groups":10,"players_in_multiple_groups":0,"duplicate_events":0,"left_in_redis_queue":0,"server_errors":0,"p50_ms":7.9,"p95_ms":19.8,"p99_ms":19.8,"max_ms":19.8,"rss_mb":191}
{"mode":"redis","players":20,"stagger_ms":5,"connect_all_ms":24.4,"queue_wall_ms":99,"matched":20,"stranded":0,"groups":10,"players_in_multiple_groups":0,"duplicate_events":0,"left_in_redis_queue":0,"server_errors":0,"p50_ms":8.8,"p95_ms":19.4,"p99_ms":19.4,"max_ms":19.4,"rss_mb":190}
{"mode":"redis","players":20,"stagger_ms":5,"connect_all_ms":24.9,"queue_wall_ms":100.1,"matched":20,"stranded":0,"groups":10,"players_in_multiple_groups":0,"duplicate_events":0,"left_in_redis_queue":0,"server_errors":0,"p50_ms":8.6,"p95_ms":19.2,"p99_ms":19.2,"max_ms":19.2,"rss_mb":191}
{"mode":"redis","players":100,"stagger_ms":5,"connect_all_ms":63.8,"queue_wall_ms":500.8,"matched":100,"stranded":0,"groups":50,"players_in_multiple_groups":0,"duplicate_events":0,"left_in_redis_queue":0,"server_errors":0,"p50_ms":7.4,"p95_ms":13.3,"p99_ms":19.9,"max_ms":19.9,"rss_mb":201}
{"mode":"redis","players":100,"stagger_ms":5,"connect_all_ms":62.8,"queue_wall_ms":3496.6,"matched":99,"stranded":1,"groups":50,"players_in_multiple_groups":1,"duplicate_events":1,"left_in_redis_queue":1,"server_errors":0,"p50_ms":6.6,"p95_ms":13.4,"p99_ms":24.8,"max_ms":24.8,"rss_mb":200}
{"mode":"redis","players":100,"stagger_ms":5,"connect_all_ms":64.5,"queue_wall_ms":3497.8,"matched":99,"stranded":1,"groups":50,"players_in_multiple_groups":1,"duplicate_events":1,"left_in_redis_queue":1,"server_errors":0,"p50_ms":7.5,"p95_ms":10.7,"p99_ms":20.8,"max_ms":20.8,"rss_mb":202}
{"mode":"redis","players":200,"stagger_ms":5,"connect_all_ms":103.4,"queue_wall_ms":1001.1,"matched":200,"stranded":0,"groups":100,"players_in_multiple_groups":0,"duplicate_events":0,"left_in_redis_queue":0,"server_errors":0,"p50_ms":7.8,"p95_ms":10.6,"p99_ms":19.6,"max_ms":19.6,"rss_mb":212}
{"mode":"redis","players":200,"stagger_ms":5,"connect_all_ms":129.3,"queue_wall_ms":3997.4,"matched":199,"stranded":1,"groups":100,"players_in_multiple_groups":1,"duplicate_events":1,"left_in_redis_queue":1,"server_errors":0,"p50_ms":7.8,"p95_ms":11.9,"p99_ms":19,"max_ms":21.1,"rss_mb":207}
{"mode":"redis","players":200,"stagger_ms":5,"connect_all_ms":101.9,"queue_wall_ms":3996.8,"matched":199,"stranded":1,"groups":100,"players_in_multiple_groups":1,"duplicate_events":1,"left_in_redis_queue":1,"server_errors":0,"p50_ms":7.4,"p95_ms":10.5,"p99_ms":13.1,"max_ms":17.3,"rss_mb":207}
```

Disclaimer

Do that that this was an extremely old rudimentary version of the scalability tests, honestly, this was my blunder (Ntu) for not being able to get it out on time, I was not able to properly manage my time with balancing it out with Math, so for now it also is just testing mainly for the math matchmaking, but the main thing that was tested is match making as well.

Note thtat there was some ai assistance but it was mainly for some seeding and formatting of this document, it is very old (Around 3 Sept) so some stuff might be slightly outdated.